14

Лекция 8–9 декабря

Схемы и верификация

Константин Чухарев

Дискретная математика — Осень 2026

Логические схемы

> *Что я не могу создать, я не понимаю.*
>
> ***— Ричард Фейнман***

Вентили

Вентиль — физический исполнитель булевой операции: два уровня напряжения на входах, один на выходе.

Вентиль	Функция	Смысл
NOT	$\overline{x}$	инвертирует
AND	$x \wedge y$	оба входа — 1
OR	$x \vee y$	хотя бы один — 1
XOR	$x \oplus y$	входы различны
NAND	$\overline{x \wedge y}$	отрицание AND
NOR	$\overline{x \vee y}$	отрицание OR

Пары читаются вопросами: AND — «оба — единицы?», OR — «хотя бы один?», XOR — «различны?», а NAND и NOR отрицают ответы AND и OR.

Таблицы истинности вентилей

Формула и таблица — два равноправных языка для одного и того же вентиля.

x	y	$x \wedge y$	$x \vee y$	$x \oplus y$	$\overline{x \wedge y}$	$\overline{x \vee y}$
0	0	0	0	0	1	1
0	1	0	1	1	1	0
1	0	0	1	1	1	0
1	1	1	1	0	0	0

NOT — единственный одновходовый вентиль набора: $\overline{0} = 1$, $\overline{1} = 0$.

OR и XOR совпадают в трёх строках из четырёх и расходятся только при $x = y = 1$.

Русское «или» бывает исключающим («чай или кофе»), вентиль OR — никогда: при двух единицах он честно выдаёт 1.

Сколько существует наборов входа?

У вентиля с n входами 2^n входных наборов — по строке таблицы на каждый.

$$n = 2 : 4, \quad n = 3 : 8, \quad n = 10 : 1024, \quad n = 64 : 2^{64} \approx 1.8 \cdot 10^{19}.$$

Каждая строка независима: её выход выбирается отдельно, нулём или единицей.

Всего n-арных булевых функций 2^{2^n} — при $n = 2$ ровно $2^4 = 16$.

Таблица растёт экспоненциально, схема может быть компактной.

Спецификация: аварийное реле

Реле аварии в серверной стойке: x — перегрев, y — вентилятор охлаждения, z — кнопка теста, и таблица пяти режимов срабатывания.

x	y	z	f
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	1

Проверяем нули: строка $(1, 1, 0)$ — стойка греется, но вентилятор работает, и реле молчит.

Из таблицы в формулу: минтермы

Формулы у реле нет — только таблица.

Чтение по строкам: кнопка теста z звонит в любом режиме, а без теста реле срабатывает только от перегрева при выключенном вентиляторе.

Строим формулу прямо по строкам.

Минтерм — конъюнкция всех переменных, истинная ровно на одной строке: на месте единицы переменная входит без отрицания, на месте нуля — с отрицанием.

Выписываем минтермы всех пяти единичных строк:

$$\overline{x}\overline{y}z, \quad \overline{x}yz, \quad x\overline{y}\overline{z}, \quad x\overline{y}z, \quad xyz.$$

Каждый минтерм — детектор своей строки: 1 только на ней и 0 на всех остальных.

Дизъюнкция минтермов — совершенная ДНФ (СДНФ): она равна 1 ровно там, где $f = 1$.

Склеивания: от СДНФ к короткой формуле

Пара минтермов, отличающихся ровно в одной переменной, склеивается — переменная исчезает.

$$\overline{x}\overline{y}z \vee \overline{x}yz = \overline{x}z, \qquad x\overline{y}\overline{z} \vee x\overline{y}z = x\overline{y}.$$

Ещё два шага в том же духе: $xyz \vee x\overline{y}z = xz$, затем $\overline{x}z \vee xz = z$.

$$f = z \vee x\overline{y}.$$

Из пяти минтермов уцелело два терма: формула снова совпала со словесной формулировкой требования.

Склеивание работает только на соседних строках — отличающихся в одной позиции.

Карта Карно — та же операция, разложенная на клетках.

Из формулы в схему

Пример (Схема для реле)

Формуле $f = z \vee x\overline{y}$ отвечает цепочка из трёх вентилей.

- NOT: $\overline{y}$ — уровень 1.
- AND: $x \wedge \overline{y}$ — уровень 2.
- OR: $z \vee (x \wedge \overline{y})$ — уровень 3.

Схема из СДНФ до склеиваний: 3 NOT + 10 AND + 4 OR — 17 вентилей, глубина 6.

Минимизация сократила её до 3 вентилей и глубины 3 — размер упал почти в шесть раз.

Проверяем вход $(1, 0, 0)$ — $\overline{y} = 1$, $x \wedge \overline{y} = 1$, реле срабатывает при перегреве и выключенном вентиляторе.

Функциональная полнота

Определение 1 (Функциональная полнота)

Набор вентилей называется **функционально полным**, если из него собирается схема для любой булевой функции.

Релейное построение уже дало ответ: НЕ, И и ИЛИ достаточно для любой таблицы.

Производство спрашивает дальше: сколько типов вентилей выпускать, проверять и держать на складе.

Каждый выброшенный из базиса вентиль — минус один тип ячейки в библиотеке стандартных элементов.

Полнота НЕ, И и ИЛИ

Теорема 1 (Полнота построением)

Набор NOT, AND, OR функционален для любой булевой функции от n переменных.

Доказательство (построением по таблице)

Приём тот же, что и для реле.

Для каждой строки с $f = 1$ выпишем минтерм, вентилями AND соберём конъюнкции, вентилями OR соединим их.

Отрицания для минтермов дают вентили NOT, по одному на переменную.

Дизъюнкция минтермов равна 1 ровно на единичных строках, поэтому собранная схема вычисляет f.

Построение даёт верхнюю оценку размера.

Шестнадцать функций двух входов

Все $2^4 = 16$ функций кодируются словами из значений на наборах $00, 01, 10, 11$: слово 0110 даёт единицы только на 01 и 10 — это XOR.

Слово	Функция	Имя
0000	0	константа 0
0001	$x \wedge y$	конъюнкция
0010	$x\overline{y}$	запрет y
0011	x	проекция x
0100	$\overline{x}y$	запрет x
0101	y	проекция y
0110	$x \oplus y$	XOR
0111	$x \vee y$	дизъюнкция

Слово	Функция	Имя
1000	$\overline{x \vee y}$	$\downarrow$ — **полон**
1001	$x \equiv y$	эквивалентность
1010	$\overline{y}$	NOT y
1011	$y \to x$	обратная импликация
1100	$\overline{x}$	NOT x
1101	$x \to y$	импликация
1110	$\overline{x \wedge y}$	$\uparrow$ — **полон**
1111	1	константа 1

Полнота одного NAND

Теорема 2 (Штрих Шеффера)

Уже один NAND позволяет собрать НЕ, И и ИЛИ:

$$\overline{x} = x \uparrow x, \quad x \wedge y = (x \uparrow y) \uparrow (x \uparrow y), \quad x \vee y = (x \uparrow x) \uparrow (y \uparrow y).$$

Доказательство

Дизъюнкция проверяется де Морганом: $(x \uparrow x) \uparrow (y \uparrow y) = \overline{\overline{x}\,\overline{y}} = x \vee y$.

Конъюнкция — двойное отрицание над NAND: $\overline{\overline{x \wedge y}} = x \wedge y$.

На битах: при $x = y = 1$ пара $1 \uparrow 1 = 0$ даёт $(1 \uparrow 1) \uparrow (1 \uparrow 1) = 0 \uparrow 0 = 1$ — конъюнкция единиц вышла единицей.

Тройка НЕ, И, ИЛИ полна, и каждая её операция выражена через NAND, — полнота наследуется.

Почему полны только NAND и NOR?

Поодиночке из шестнадцати функций полны ровно две — NAND (второе имя: штрих Шеффера, *Sheffer stroke*) и NOR.

Остальные проваливают критерий Поста: константы и проекции тривиальны, AND монотонен, XOR линеен, импликация сохраняет единицу.

> В КМОП-технологии NAND собирается из четырёх транзисторов, а AND — из шести: инвертор на выходе.
>
> Поэтому промышленные библиотеки стандартных ячеек строятся вокруг NAND.

Схема как граф

Определение 2 (Логическая схема)

Схема — ориентированный ациклический граф: истоки — входы, внутренние узлы — вентили, стоки — выходы.

Выходы одних вентилей подаются на входы других — схема вычисляет композицию булевых функций.

Релейная схема — такой граф: истоки x, y, z, три вентиля, один сток f.

Ацикличность — комбинационная логика: результат зависит только от текущих входов.

Цикл в схеме сделал бы значение неопределённым: выход ждал бы сам себя.

Размер и глубина

Определение 3 (Размер и глубина)

Размер схемы — число вентилей: площадь, стоимость.

Глубина — длина самого длинного пути от входа к выходу: задержка.

У реле числа уже посчитаны: размер 3 (NOT, AND, OR), глубина 3 — вентили стоят друг за другом.

Размер и глубина — базовый инженерный компромисс.

Схема с малой глубиной собирается из большего числа вентилей, а компактная работает медленнее.

* Цена СДНФ: верхняя оценка

Теорема 3 (Схема из таблицы)

Функция от n переменных, равная единице на k наборах, вычисляется схемой размера не более

$$n + (n-1)k + (k-1).$$

Доказательство

Считаем вентили СДНФ-схемы: отрицаний не больше n — по одному на переменную с веером на все минтермы, конъюнкций $(n-1)k$, дизъюнкций $k-1$.

При максимальном $k = 2^n$ оценка даёт $(n+1) \cdot 2^n$ — экспоненту от числа входов.

Реле — точно этот случай: $n = 3$, $k = 5$, и $3 + 2 \cdot 5 + 4 = 17$ — те же семнадцать вентилей, что вышли прямым счётом.

* Чётность: экспонента против цепочки

Пример (Чётность n бит)

Функция равна 1, когда единиц во входе чётное число.

Единичных наборов половина — 2^{n-1}, любые два различаются минимум в двух битах, поэтому склеивать нечего: минимальная ДНФ чётности — её СДНФ.

Порядок размера: $n \cdot 2^{n-1}$ вентилей против цепочки из $n-1$ XOR — при $n=32$ это $6.9 \cdot 10^{10}$ против тридцати одного.

Одна функция — экспонента в базисе НЕ, И, ИЛИ и линейная схема в базисе с XOR.

Цена функции зависит и от самой функции, и от того, какие вентили объявлены базисом.

Задержка: от глубины к такту

Модель задержек: каждый вентиль переключается за фиксированное время τ, провода считаем мгновенными.

Задержка сигнала — глубина, умноженная на τ, а самый длинный путь называется **критическим**.

Такт процессора не короче худшего комбинационного пути — тактовая частота ограничена глубиной.

Схема XOR имеет глубину 3 — три задержки τ по пути AND — NOT — AND.

Схема для XOR

Пример

$x \oplus y = (x \vee y) \wedge \overline{x \wedge y}$.

Читается по смыслу: AND ловит «оба — единицы», NOT даёт «не оба», OR собирает «хотя бы один», финальный AND требует обоих.

Вентилей четыре — OR, AND, NOT, AND: размер 4, а самый длинный путь $x, y \to$ AND $\to$ NOT $\to$ AND даёт глубину 3.

Подадим $x = y = 1$: $\overline{x \wedge y} = 0$ и выход 0 — расхождение с OR.

Подадим $x = 1$, $y = 0$: запрет снят, OR даёт 1, выход 1.

Задержку задаёт критический путь — не сумма всех вентилей.

Параллельные ветви считаются одновременно.

* Семейства схем

Одна схема имеет фиксированное число входов: схема для 4-битного сложения не сложит 8-битные числа.

Определение 4 (Семейство схем)

Семейство схем — последовательность $C_1, C_2, \ldots$, где C_n имеет n входов.

Семейство **равномерно**, если существует алгоритм, строящий C_n по n.

Чётность: C_n — цепочка из $n - 1$ XOR-вентилей.

Равномерные полиномиальные семейства — схемный взгляд на эффективные вычисления.

Полусумматор

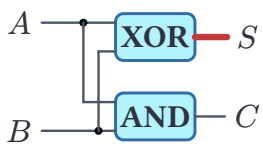
Определение 5 (Полусумматор (*half adder*))

Складывает два бита A и B:

$$S = A \oplus B, \quad C = A \wedge B.$$

Каждый выход отвечает за свой разряд столбика: S — младший, бит суммы по модулю 2, поэтому его даёт XOR.

C — старший, и он равен 1 только при $A = B = 1$: лишь там сумма переливается через разряд, поэтому его даёт AND.



Полусумматор: таблица

A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Нижняя строка — вся механика переноса: $1 + 1 = 10_2$, младший бит уходит в S, старший — в C.

> «Полу-» — честно: столбик при сложении складывает три бита, а эта схема — только два.
>
> Входящего переноса у полусумматора нет.

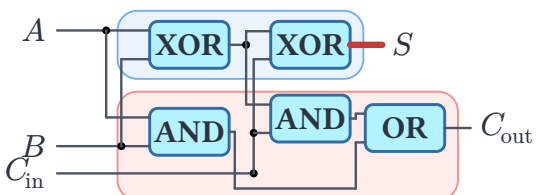
Полный сумматор

Определение 6 (Полный сумматор (*full adder*))

Складывает три бита — A, B и входящий перенос C_{in}:

$$S = A \oplus B \oplus C_{\text{in}}, \quad C_{\text{out}} = (A \wedge B) \vee (C_{\text{in}} \wedge (A \oplus B)).$$

Подаются три бита, снимаются два: бит суммы — чётность входов, а перенос уходит в старший разряд.



Чтение схемы: перенос по вентилю

Обратная дорога — из схемы в формулу: имя каждого провода есть формула его вентиля.

Читаем схему переносов слева направо, снизу вверх:

- XOR и AND на входах: $u = A \oplus B$, $v = A \wedge B$ — уровень 1.
- Второй AND: $w = C_{\text{in}} \wedge u$ — уровень 2.
- OR на выходе: $C_{\text{out}} = v \vee w$ — уровень 3.

Подстановка снизу вверх разворачивает схему в одну формулу:

$$C_{\text{out}} = (A \wedge B) \vee (C_{\text{in}} \wedge (A \oplus B)).$$

Проверяем вход $(A, B, C_{\text{in}}) = (0, 1, 1)$ — $u = 1$, $v = 0$, $w = 1$, перенос равен 1, как и положено при $0 + 1 + 1 = 10_2$.

Схема и формула — два представления одной функции, и оба обратимы.

СДНФ переводит таблицу в схему, чтение по проводам — схему в формулу.

Функция переноса как большинство

Перенос $C_{\text{out}} = 1$ тогда и только тогда, когда хотя бы два из трёх входов равны 1.

Перенос — это уже знакомая функция большинства.

Совпадение с формулой схемы проверяется раскрытием $\oplus$ и поглощением:

$$(A \wedge B) \vee (C_{\text{in}} \wedge (A \oplus B)) = (A \wedge B) \vee (C_{\text{in}} \wedge A) \vee (C_{\text{in}} \wedge B).$$

Справа — большинство, выписанное напрямую: три конъюнкции, по одной на каждую пару входов.

На битах: при $(0, 1, 1)$ единственная полная пара даёт $C_{\text{in}} \wedge B = 1$ — перенос горит, а при $(0, 1, 0)$ не собрана ни одна пара — перенос погас.

Формулы полусумматора и полного сумматора — минимизированные ДНФ.

Из минимизированных формул собирается арифметика процессора.

Каскад сумматоров

Определение 7 (Ripple-carry)

Цепочка из n полных сумматоров: перенос каждого разряда подаётся на вход следующего.

Размер $O(n)$, глубина $O(n)$.

Пример

Сложим $0111 + 0001$ и проследим перенос по разрядам.

Разряд 0: $1 + 1 + 0 = 10_2$ — бит 0, перенос 1.

Разряды 1 и 2: снова $1 + 0 + 1 = 10_2$ — бит 0, перенос идёт дальше.

Разряд 3: $0 + 0 + 1 = 01_2$ — бит 1, перенос 0, волна гаснет.

Результат 1000 — перенос прошёл все четыре разряда, и каждый ждал соседа.

Ripple-carry в вентилях

Пример (Точный счёт)

Полный сумматор — пять вентилей: два XOR для суммы, два AND и OR для переноса.

Внутри одного разряда перенос проходит AND и OR — две задержки.

Первый перенос готов за 3 задержки, каждый следующий — ещё за 2, финальный — за $2n + 1$.

Итог для n бит — размер $5n$ и глубина $2n + 1$.

4 бита — 20 вентилей и глубина 9, 64 бита — 320 вентилей и глубина 129.

Оценка $O(n)$ из определения развернулась в точные числа: 5 и 2 — цена одного разряда.

Каждое удвоение разрядности удваивает и площадь, и задержку.

Глубина каскада

Глубина каскада ограничивает тактовую частоту: такт не короче критического пути.

Опережающий перенос (*carry-lookahead*) считает переносы группами и снижает глубину до $O(\log n)$ — ценой роста размера.

Вычитание тоже каскад: $A - B = A + \overline{B} + 1$ в дополнительном коде.

Вся арифметика процессора — вариации одного сумматора.

Работа и глубина

За сложением стоит общий торг двух ресурсов.

Работа — суммарное число вентильных операций, у схемы это размер: площадь, энергия.

Глубина — параллельное время: сколько уровней логики обязан пройти сигнал.

Каскад для сложения: работа $5n$ при глубине $2n + 1$ — минимум кремния, максимум ожидания.

Дерево переноса считает разряды группами: глубина падает до $O(\log n)$, работа растёт.

Цепочка экономит площадь, дерево покупает время — получить и то и другое сразу не выходит.

> Тот же выбор стоит в программе: один цикл или разделяй-и-властвуй под пул потоков.
>
> Параллелизм всегда оплачивается дублированием работы.

Код: глубина нетлиста

Нетлист — схема в тексте: список вентилей в топологическом порядке.

```
# Нетлист реле: f = z or (x and not y).
gates = [("not", "y"), ("and", "x", "g1"), ("or", "z", "g2")]
d = {v: 0 for v in "xyz"}  # входы --- уровень 0
for i, (_, *ins) in enumerate(gates):
    d[f"g{i + 1}"] = 1 + max(d[w] for w in ins)
print(d["g3"])  # 3
```

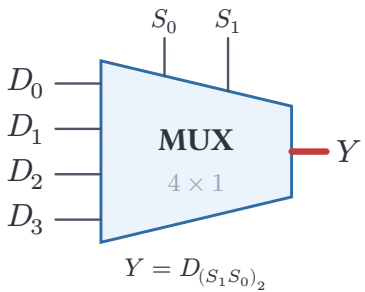
Один проход считает уровни всех проводов: уровень вентиля — максимум уровней его входов плюс один.

Тот же проход делают симуляторы схем, а для SAT нетлист переводится в КНФ.

* АЛУ

Арифметико-логическое устройство — вычислительное ядро процессора.

Однобитный срез: блоки AND, OR, сложения, вычитания считают на одних и тех же A, B, а мультиплексор по адресу $S_1 S_0 = 10$ выводит на Y выбранный D_2.



Мультиплексор «из 2^k в 1» выдаёт один из 2^k входов по k адресным битам.

n-битное АЛУ — n таких срезов с цепочкой переносов.

Код Грея

Определение 8 (Код Грея (*Gray code*))

Упорядочение всех 2^n двоичных строк длины n, где соседние строки — включая последнюю и первую — отличаются ровно в одном бите.

Пример

G_3: $000, 001, 011, 010, 110, 111, 101, 100$.

Каждая соседняя пара — один бит разницы.

Код цикличен: последняя и первая строки тоже соседи.

Зачем код Грея?

При переключении нескольких битов сразу возникают ложные промежуточные состояния.

Двоичный счётчик на переходе 0111 → 1000 меняет все четыре бита, и энкодер успевает прочитать 0101 или 1111 — значения, которых не было никогда.

Код Грея меняет один бит за шаг — энкодеры угла и позиции читаются безошибочно.

Код Грея уже работал в карте Карно: соседние клетки — соседние строки.

Построение кода Грея

Теорема 4 (Рекурсивное построение)

$G_1 = [0, 1]$.

$$G_{n+1} = 0G_n, 1G_n^R,$$

где $0G_n$ — слова G_n с приписанным слева нулём, а $1G_n^R$ — слова G_n в обратном порядке с приписанной слева единицей.

Пример

Из $G_1 = [0, 1]$: половины $0G_1 = [00, 01]$ и $1G_1^R = [11, 10]$ — вторая обращена и приписана с единицей слева.

Стык половин отличается только первым битом — обращение второй половины и даёт цикличность.

Перевод между кодами

Теорема 5 (Перевод через XOR)

Двоичный код в код Грея: $g_i = b_i \oplus b_{i+1}$, старший бит сохраняется.

Обратно: $b_i = g_i \oplus b_{i+1}$ — накопительный $\oplus$ от старшего бита.

Пример

$b = 1011$: $g = 1110$ — старший бит тот же, дальше $\oplus$ соседей.

Обратно по $g = 1110$: $b_3 = 1$, $b_2 = 1 \oplus 1 = 0$, $b_1 = 1 \oplus 0 = 1$, $b_0 = 0 \oplus 1 = 1$ — вернулись к исходному 1011.

Оба преобразования — цепочка XOR-вентилей глубины $O(n)$.

Верификация через SAT

> *Тестирование программы может убедительно продемонстрировать наличие ошибок, но безнадёжно несостоятельно для демонстрации их отсутствия.*
>
> ***— Эдсгер Дейкстра***

Задача верификации

Схема спроектирована, затем оптимизирована — и нужно доказать, что поведение не изменилось.

Перебор всех входов невозможен: у n входов 2^n наборов.

Обе схемы переводят в КНФ: на каждый вентиль заводится переменная его выхода и дизъюнкты, задающие таблицу вентиля (преобразование Цейтина).

Размер растёт линейно по числу вентилей.

Митер

Определение 9 (Митер (*miter*))

Соединим входы схем C_1 и C_2 попарно, на каждую пару выходов поставим XOR, а выходы всех XOR соберём одним OR.

Митер выдаёт 1 ровно на тех входах, где схемы различаются: XOR срабатывает на расхождении пары выходов, OR собирает любые расхождения вместе.

Теорема 6 (Эквивалентность через невыполнимость)

Схемы C_1 и C_2 эквивалентны тогда и только тогда, когда их митер невыполним.

Митер в действии

Пример

Сравним $f_1(x, y) = x \vee y$ и $f_2(x, y) = x \oplus y$.

На входе $(1, 1)$: $f_1 = 1$, $f_2 = 0$ — XOR выходов даёт 1, митер срабатывает.

На входе $(1, 0)$ обе функции выдают 1 — расхождения нет.

А схема $(x \vee y) \wedge \overline{x \wedge y}$ эквивалентна вентилю XOR: их митер невыполним, и решатель доказал бы это без проверки наборов.

На двух входах хватило бы перебора, но у промышленных схем тысячи входов: 2^n наборов никто не перебирает, а UNSAT-ответ остаётся доказательством.

SAT — решатель предъявляет контрпример: вход, разводящий схемы.

UNSAT — доказательство эквивалентности на всех входах сразу.

Единичное распространение

Определение 10 (Unit propagation)

Если в дизъюнкте все литералы, кроме одного, ложны, оставшийся обязан быть истинным.

Пример (Каскад без ветвлений)

$(x) \wedge (\overline{x} \vee y) \wedge (\overline{y} \vee z) \wedge \overline{z}$.

Единичный дизъюнкт (x) даёт $x = 1$ — дизъюнкт $(\overline{x} \vee y)$ становится единичным: $y = 1$.

Затем $z = 1$ — и последний дизъюнкт ложен.

Формула невыполнима, противоречие найдено без единого ветвления.

Дизъюнкт как импликация

Дизъюнкт из двух литералов равносилен паре импликаций:

$$l_1 \vee l_2 \quad \equiv \quad \overline{l_1} \to l_2, \quad \overline{l_2} \to l_1.$$

Вторая импликация — контрапозиция первой.

Потерять её — потерять половину рёбер будущего графа.

Тройку литералов так разложить нельзя — граница между лёгким и трудным случаями SAT.

Граф импликаций

Определение 11 (Граф импликаций)

Вершины — все литералы: x и $\overline{x}$ для каждой переменной.

Рёбра — обе импликации каждого дизъюнкта.

Выполнимость превращается в вопрос о достижимости в графе.

Путь в графе — цепочка принуждений: истинность начала пути тянет за собой истинность конца.

Поэтому конфликтом грозит именно цикл, собирающий пару $x, \overline{x}$.

Критерий 2-SAT

Теорема 7 (Критерий)

Формула 2-SAT невыполнима тогда и только тогда, когда x и $\overline{x}$ лежат в одной компоненте сильной связности графа импликаций.

Доказательство (в обе стороны)

Если x и $\overline{x}$ взаимно достижимы, импликации дают $x \to \overline{x}$ и $\overline{x} \to x$ — противоречие при любом значении x.

Если же пары разнесены по компонентам, присваивание по обратному топологическому порядку компонент делает каждую импликацию истинной.

К компонентам сильной связности приводит обход в глубину за $O(n + m)$.

Построение выполняющего набора

Если пары разнесены по компонентам, модель читается из графа.

1. Найти компоненты сильной связности и топологический порядок их конденсации.
2. Обрабатывать компоненты в обратном порядке — от стоков к истокам.
3. Переменная получает 1, если её компонента идёт позже компоненты отрицания, иначе 0.

Обратный порядок гарантирует совместимость: к моменту решения о литерале все зависимые от него литералы уже получили значения.

По пути в графе значение не убывает: ложь перед истиной.

Поэтому каждая импликация оказывается истинной.

Граф импликаций: пример

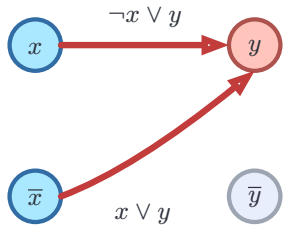


Рис. 1. Передние рёбра, контрапозиции опущены.

Пример

Формула $(x \vee y) \wedge (\overline{x} \vee y)$: оба дизъюнкта ведут в y, и пары $x, \overline{x}$ и $y, \overline{y}$ лежат в разных компонентах — формула выполнима.

Из графа видно значение: оба пути требуют истинности y.

$(x \vee y)$ даёт $\overline{x} \to y$, $(\overline{x} \vee y)$ даёт $x \to y$ — оба требования сходятся в y.

2-SAT: полный экземпляр

Пример (Три дизъюнкта, шесть рёбер)

$\varphi = (x \vee y) \wedge (\overline{x} \vee z) \wedge (\overline{y} \vee \overline{z})$.

Рёбра: $\overline{x} \to y$, $\overline{y} \to x$, $x \to z$, $\overline{z} \to \overline{x}$, $y \to \overline{z}$, $z \to \overline{y}$.

Циклы собираются в две компоненты: $\{\overline{x}, y, \overline{z}\}$ и $\{x, z, \overline{y}\}$ — ни одна не содержит пары $x, \overline{x}$.

Формула выполнима, и модель читается прямо из компонент: $x = 1$, $z = 1$, $y = 0$.

Подстановка даёт $(1 \vee 0) = 1$, $(0 \vee 1) = 1$, $(1 \vee 0) = 1$ — все три дизъюнкта истинны.

2-SAT против 3-SAT

2-SAT решается за линейное время.

3-SAT — SAT, где каждый дизъюнкт содержит ровно три литерала, — NP-полна.

Один лишний литерал — скачок сложности.

Причина структурная: пара литералов распадается на импликации, тройка — нет.

Длинный дизъюнкт режут на тройки свежими переменными (преобразование Цейтина), но в 2-SAT формула от этого проще не становится. Промышленные решатели берут общий SAT методом CDCL: на каждом конфликте учится новый дизъюнкт, а эвристика ведёт выбор переменных — так работают MiniSat и z3.

Хорновские дизъюнкты

Определение 12 (Хорновский дизъюнкт)

Хорновский дизъюнкт (*Horn clause*) — дизъюнкт с не более чем одним положительным литералом:

$$\overline{p_1} \vee \ldots \vee \overline{p_k} \vee q.$$

Эквивалентная запись — правило: $p_1 \wedge \ldots \wedge p_k \to q$.

Три роли одного дизъюнкта: правило с посылками — «если, то», факт — данные, цель — запрос.

При $k = 0$ правило без посылок — факт q. Дизъюнкт без положительных литералов — цель.

Хорн: примеры

Пример

$(\overline{x} \vee \overline{y} \vee z)$ — правило $x \wedge y \to z$.

$(\overline{x} \vee y)$ — правило $x \to y$.

(x) — факт, а $(\overline{x} \vee \overline{y})$ — цель «не оба сразу».

Хорновские формулы — язык правил: база знаний состоит из фактов и правил вывода.

На них стоит логическое программирование, а вывод типов в компиляторах — решение хорновской системы ограничений.

Хорн: линейное время

Теорема 8 (Хорн-SAT линеен)

Выполнимость хорновской формулы решается за время, линейное по суммарной длине её дизъюнктов.

Распространение срабатывает по разу на дизъюнкт — когда истинной становится его последняя посылка:

1. Все факты пометить истинными и положить в очередь.
2. Правило с истинными посылками порождает вывод: пометить и в очередь.
3. Опустевшая цель — пустой дизъюнкт: невыполнимость.

Определение 13 (Наименьшая модель)

Модель, истинная ровно на принуждённых литералах, называется **наименьшей моделью** формулы. У выполнимой хорновской формулы она одна.

Хорн: прогон

Пример (Прогон)

$\varphi = (x) \wedge (\overline{x} \vee y) \wedge (\overline{z} \vee w)$.

Факт x принуждает y из правила $x \to y$, и распространение заканчивается.

Переменные z и w никем не принуждаются: наименьшая модель $x = 1, y = 1, z = 0, w = 0$.

Добавьте цель $(\overline{x} \vee \overline{y})$ — после $x = 1, y = 1$ она пустеет: UNSAT.

Распространение строит **наименьшую модель**: истинным объявляется только принуждённое.

Семантика Пролога опирается именно на наименьшую модель — поэтому Хорн и линеен, и предсказуем.

CDCL

Современные решатели — CDCL (*Conflict-Driven Clause Learning*): DPLL плюс обучение на конфликтах.

- решение: эвристика VSIDS выбирает переменную — приоритет у частых участников недавних конфликтов.
- анализ конфликта: по истории присваиваний извлекается выученный дизъюнкт.
- нехронологический возврат и периодические перезапуски.

DPLL при конфликте просто отступает.

CDCL запоминает его причину — и не ходит туда второй раз.

Выученный дизъюнкт

Пример (Конфликт и резолюция)

Пусть решения дали $x_4 = 1$ и $x_5 = 1$.

Дизъюнкты $(\overline{x_4} \vee \overline{x_5} \vee x_6)$ и $(\overline{x_4} \vee \overline{x_5} \vee \overline{x_6})$ ложны независимо от x_6 — конфликт.

Резолюция двух дизъюнктов по x_6 даёт **выученный дизъюнкт**:

$$\overline{x_4} \vee \overline{x_5}.$$

Комбинация $x_4 = x_5 = 1$ запрещена до конца поиска.

Выученные дизъюнкты — шаги резолюции.

Каждый отсекает целое подпространство перебора, а не одну ветвь.

Сертификат невыполнимости

Ответ UNSAT тоже предъявляет доказательство: выученные дизъюнкты — шаги резолюции.

Решатель сохраняет цепочку резолюций, приведшую к пустому дизъюнкту.

Независимая проверяющая программа воспроизводит её за доли секунды.

Верить решателю не нужно: доказательство проверяется механически.

Для митера доказательством эквивалентности схем служит невыполнимость формулы различий.

DPLL: правила

DPLL спускается по формуле двумя правилами, и оба сохраняют выполнимость: вычёркивается только заведомо невыполнимое.

Определение 14 (Единичное распространение)

Единичный дизъюнкт — дизъюнкт, у которого все литералы, кроме одного, уже ложны.

Оставшийся литерал обязан быть истинным: присваиваем его и повторяем до неподвижной точки или конфликта.

Определение 15 (Чистый литерал)

Литерал **чист**, если его отрицание не встречается в формуле.

Чистому литералу можно смело дать единицу: он не опровергает ни один дизъюнкт, и его дизъюнкты вычёркиваются целиком.

DPLL: ветвление и конфликт

Когда оба правила исчерпаны, а дизъюнкты остались, формула требует выбора — и поиск ветвится.

Определение 16 (Ветвление)

Выбирается неприсвоенная переменная, и оба её значения опробуются по очереди.

Конфликт — пустой дизъюнкт: ветвь закрывается, и поиск возвращается к последнему выбору.

Оба правила не теряют выполнимость: единичный литерал принуждён, чистому годится единица.

Поэтому исчерпание всех ветвей — честное доказательство невыполнимости, а не каприз перебора.

DPLL: прогон

Пример (Формула из четырёх дизъюнктов)

$$\varphi = (x \vee y) \wedge (\overline{x} \vee y) \wedge (x \vee \overline{y}) \wedge (y \vee z).$$

Единичных дизъюнктов нет, зато z — чистый литерал.

Отступы в трассе — глубина ветвления:

```
ветвим 1 = False
  unit   2
  конфликт
ветвим 1 = True
  unit   2
  модель {1: True, 2: True}
```

Ветвь $x = 0$ умерла на $(x \vee \overline{y})$, ветвь $x = 1$ выполнила всё, и z остался свободен.

DPLL: код

Ядро умещается в полтора десятка строк: литерал — целое со знаком, 1 — это x, -1 — это $\overline{x}$.

```
def step(clauses, lit):
    return [[l for l in c if l != -lit] for c in clauses if lit not in c]

def dpll(clauses, model):
    while True:  # единичное распространение
        unit = next((c[0] for c in clauses if len(c) == 1), None)
        if unit is None: break
        print("unit  ", unit)
        model[abs(unit)] = unit > 0
        clauses = step(clauses, unit)
        if [] in clauses:
            print("конфликт")
            return None
```

DPLL: код, ветвление

```
    if not clauses: print("модель", model); return model
    lit = clauses[0][0]
    for val in (False, True):  # ветвление
        print("ветвим", lit, "=", val)
        r = dpll(step(clauses, lit if val else -lit), {**model, abs(lit): val})
        if r is not None: return r
    return None

dpll([[1, 2], [-1, 2], [1, -2], [2, 3]], {})  # (x y) (-x y) (x -y) (y z)
```

Трассировку при желании можно обогатить чистыми литералами: тройка строк с их поиском встанет перед ветвлением.

Индустриальные решатели добавляют к этому каркасу выученные дизъюнкты — так из DPLL вырастает CDCL.

Кодирование: раскраска графа

Определение 17 (k-раскраска)

***k*-раскраска** графа $G = (V, E)$ — функция $V \to \{1, ..., k\}$, переводящая смежные вершины в разные цвета.

Переменные: $x_{v,c} = 1$, если вершина v получает цвет c.

Ограничения:

- у каждой вершины есть цвет: $\bigvee_{c=1}^{k} x_{v,c}$.
- не более одного цвета: $\overline{x_{v,c}} \vee \overline{x_{v,d}}$ для каждой пары $c < d$.
- смежные различаются: $\overline{x_{u,c}} \vee \overline{x_{v,c}}$ для каждого ребра и цвета.

Всего $O(|V| \cdot k^2 + |E| \cdot k)$ дизъюнктов — полином от размера графа, и решатель получает экземпляр своей родной формы.

Треугольник в два цвета

Пример (Полный пример)

Граф — треугольник: вершины a, b, c попарно смежны, цветов $k = 2$.

Дизъюнкты «у каждой вершины есть цвет»:

$$\left(x_{a,1} \vee x_{a,2}\right) \wedge \left(x_{b,1} \vee x_{b,2}\right) \wedge \left(x_{c,1} \vee x_{c,2}\right).$$

Дизъюнкты рёбер — по одному на каждый цвет:

$$\left(\overline{x_{a,1}} \vee \overline{x_{b,1}}\right) \wedge \left(\overline{x_{a,1}} \vee \overline{x_{c,1}}\right) \wedge \left(\overline{x_{b,1}} \vee \overline{x_{c,1}}\right) \wedge$$
$$\left(\overline{x_{a,2}} \vee \overline{x_{b,2}}\right) \wedge \left(\overline{x_{a,2}} \vee \overline{x_{c,2}}\right) \wedge \left(\overline{x_{b,2}} \vee \overline{x_{c,2}}\right).$$

Попарные запреты «не более одного цвета» опустим: для противоречия они не нужны.

Треугольник: вывод

Пример

Цветов два, вершин три: какие-то две вершины получили один цвет.

Эти две смежны, и их рёберный дизъюнкт запрещает совпадение.

Формула невыполнима — треугольник нельзя раскрасить в два цвета.

Ответ UNSAT — доказательство несуществования.

Ответ SAT — готовая раскраска: модель формулы читается как решение задачи.

Судоку и N-ферзи

- Судоку: $x_{r,c,d}$ — «в клетке (r, c) стоит цифра d». $9^3 = 729$ переменных, около девяти тысяч дизъюнктов — решатель справляется за миллисекунды.
- N-ферзи: $x_{i,j}$ — «ферзь в клетке (i, j)». При $N = 8$ — порядка семисот дизъюнктов.

Решатель не знает правил игры: ему дан плоский список дизъюнктов.

Хорошая кодировка делает структуру задачи частью самих дизъюнктов.

SAT и NP

Определение 18 (NP-полнота)

NP — задачи, чей ответ «да» проверяется за полиномиальное время по предъявленному сертификату.

Задача **NP-полна**, если она лежит в NP и к ней полиномиально сводится любая задача из NP.

Для SAT сертификат — выполняющий набор: подставить и проверить каждый дизъюнкт.

Проверить легко, найти — перебор.

Теорема Кука—Левина

Теорема 9 (Кук — Левин, 1971)

SAT NP-полна.

Конструкция кодирует таблицу вычисления машины Тьюринга в КНФ.

Формула выполнима тогда и только тогда, когда машина принимает вход — любая NP-задача превращается в SAT.

Следствие: полиномиальный алгоритм для SAT существовал бы только при $\mathrm{P} = \mathrm{NP}$, поэтому судьба индустрии решателей висит на главном открытом вопросе сложности.

Практика против худшего случая

NP-полнота — свойство худшего случая.

Индустриальные формулы структурны, и CDCL решает формулы с миллионами переменных.

SAT — рабочий инструмент верификации, планирования и криптоанализа.

Эквивалентность схем с миллионами вентилей — стандартный шаг потока проектирования процессоров.

Теорема Кука обещала трудность, практика нашла структуру.

Что мы поняли?

- Схема — булева функция в железе: размер — площадь, глубина — задержка.
- Любая таблица переводится в схему: минтермы дают СДНФ, склеивания её сжимают.
- Полны сами по себе только NAND и NOR, остальные отсеивает критерий Поста.
- Полный сумматор: сумма — $\oplus$ трёх битов, перенос — функция большинства.
- Каскад — $5n$ вентилей и глубина $2n + 1$, дерево переноса снижает глубину до $O(\log n)$.
- Код Грея меняет один бит за шаг — энкодеры и карта Карно живут на нём.

- Митер сводит эквивалентность схем к SAT: UNSAT — эквивалентны, SAT — контрпример.
- 2-SAT линеен на графе импликаций, Хорн — на распространении фактов, DPLL — на единичных и чистых литералах.
- SAT NP-полна, но CDCL на структурных формулах решает миллионы переменных.

Вопросы для самопроверки

Вопросы для самопроверки

1. Постройте схему для $\overline{x \wedge y} \vee z$ и посчитайте её размер и глубину.
2. Запишите формулы полусумматора и полного сумматора.
3. Почему глубина каскада сумматоров ограничивает тактовую частоту?
4. Постройте G_4 из G_3 рекурсивно.
5. Переведите 0110 из кода Грея в двоичный.
6. Что выдаст SAT-решатель для митера схем $x \vee y$ и $x \oplus y$?
7. Проверьте по графу импликаций выполнимость $(x \vee y) \wedge (\overline{x} \vee y) \wedge \overline{y}$.

Самопроверка: дополнительный материал

Вопросы для самопроверки

1. Проследите DPLL на формуле $(x \vee y) \wedge (\overline{x} \vee y) \wedge (x \vee \overline{y})$ — где ветвление и каков ответ?
2. Как в хорновской формуле распространение фактов строит наименьшую модель Пролога?
3. Почему дизъюнкт из трёх литералов не разлагается в пару импликаций?
4. Почему выученный дизъюнкт отсекает больше, чем простой откат?
5. Выразите $x \vee y$ через одни вентили NAND и посчитайте, сколько их ушло.
6. Почему минимальная ДНФ чётности не сжимается склеиваниями?

Что почитать

- m11 — «Логические вентили и схемы», «Сумматоры», «Код Грея».
- m14 — «CDCL-солверы», «SAT и NP».